



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

título del TFG



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 11 de noviembre
de 2025

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Álvaro Pérez Mella, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 11 de noviembre de 2025

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	iv
Índice de tablas	v
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Secciones	5
3.2. Referencias	5
3.3. Imágenes	6
3.4. Listas de ítems	6
3.5. Tablas	7
4. Técnicas y herramientas	9
4.1. Herramientas fase de pruebas	9
5. Aspectos relevantes del desarrollo del proyecto	15
6. Trabajos relacionados	17
7. Conclusiones y Líneas de trabajo futuras	19
Bibliografía	21

Índice de figuras

3.1. Autómata para una expresión vacía	6
--	---

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	7
4.1. Comparación general entre MediaPipe y YOLO	11

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

Este apartado explica de forma precisa y concisa cuales son los objetivos que se persiguen con la realización del proyecto. Se puede distinguir entre los objetivos marcados por los requisitos del software a construir y los objetivos de carácter técnico que plantea a la hora de llevar a la práctica el proyecto.

3. Conceptos teóricos

En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Algunos conceptos teóricos de L^AT_EX¹.

3.1. Secciones

Las secciones se incluyen con el comando `section`.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.2. Referencias

Las referencias se incluyen en el texto usando `cite` [[11](#)]. Para citar webs, artículos o libros [[6](#)], si se desean citar más de uno en el mismo lugar [[1](#), [6](#)].

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz

3.3. Imágenes

Se pueden incluir imágenes con los comandos standard de $\text{\LaTeX}$, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:



Figura 3.1: Autómata para una expresión vacía

3.4. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

■

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

3.5. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

4. Técnicas y herramientas

En este capítulo se describen las herramientas principales empleadas o consideradas durante la fase de pruebas del proyecto, centradas en el ámbito del procesamiento de imágenes y la visión por computador. Se presentan de forma general sus características, finalidad y fuentes oficiales de documentación.

4.1. Herramientas fase de pruebas

Durante la fase de pruebas del proyecto se han evaluado diversas herramientas para el procesamiento y análisis de imágenes. A continuación se describen las dos más relevantes: **MediaPipe** y **YOLO**, ambas ampliamente utilizadas en el campo de la visión artificial.

MediaPipe

MediaPipe es un marco de trabajo de código abierto desarrollado por Google para la creación de *pipelines* o flujos de procesamiento multimedia en tiempo real [7, 4, 5].

Permite construir sistemas modulares basados en grafos donde cada componente (denominado *calculator*) realiza una tarea específica, como la captura de imágenes, el preprocesamiento, la inferencia de modelos de aprendizaje automático o el postprocesado de resultados. Su diseño facilita la reutilización de módulos y la integración de modelos de visión por computador en distintas plataformas (Android, iOS, escritorio o web).

Entre las soluciones más utilizadas que ofrece se incluyen:

- Detección de rostro (*Face Detection*) y malla facial 3D (*Face Mesh*).
- Detección y seguimiento de manos (*Hand Tracking*).
- Estimación de pose corporal completa (*Pose*).
- Solución integral *Holistic*, que combina rostro, manos y cuerpo.

Estas funcionalidades permiten aplicar MediaPipe a proyectos de reconocimiento facial, análisis de movimiento, interacción gestual, realidad aumentada y aplicaciones educativas o deportivas.

Toda la documentación oficial se encuentra disponible en la guía técnica de Google AI Edge [4], y el código fuente está publicado en su repositorio oficial de GitHub [5].

YOLO (You Only Look Once)

YOLO es una familia de modelos de detección de objetos en tiempo real creada por la empresa Ultralytics. Su principio básico consiste en dividir una imagen en una cuadrícula y predecir simultáneamente las coordenadas y la clase de los objetos presentes en cada región [10, 9].

La versión más reciente, YOLOv8, incorpora arquitecturas modernas optimizadas para precisión y velocidad, con soporte para varias tareas de visión artificial:

- Detección de objetos (*object detection*).
- Segmentación de instancias (*instance segmentation*).
- Clasificación de imágenes.
- Estimación de pose humana.

Se distribuye como paquete Python mediante la librería *ultralytics*, permitiendo la carga de modelos pre entrenados o el entrenamiento personalizado sobre conjuntos de datos propios.

La documentación y ejemplos de uso están disponibles en el sitio oficial de Ultralytics [10], mientras que el código fuente se mantiene en su repositorio público de GitHub [9].

Aspecto	MediaPipe	YOLO
Enfoque principal	Procesamiento de características humanas (rostro, manos, pose).	Detección y localización de objetos genéricos en imágenes o vídeo.
Tipo de modelo	Framework modular con soluciones preentrenadas de Google.	Red neuronal convolucional profunda (CNN) optimizada para detección.
Ejecución	En tiempo real, incluso en dispositivos móviles o CPU.	En tiempo real, preferentemente con GPU para rendimiento óptimo.
Configuración	Bajo nivel de configuración: soluciones listas para usar.	Permite entrenamiento y ajuste fino sobre datasets personalizados.
Precisión en personas	Alta precisión en puntos clave del cuerpo humano.	Alta precisión en la localización de personas y objetos.
Lenguaje y soporte	Implementación principal en Python y C++.	Implementación en Python con soporte mediante la librería Ultralytics.
Licencia	Apache License 2.0 (Google).	AGPL-3.0 License (Ultralytics).
Uso ideal	Análisis de movimiento, reconocimiento facial, interfaces gestuales.	Detección general de objetos, vigilancia, control de tráfico o inventario.

Tabla 4.1: Comparación general entre MediaPipe y YOLO

Comparativa entre MediaPipe y YOLO

Aunque ambos frameworks se utilizan en el ámbito de la visión por computador, su enfoque y finalidad son distintos y, en muchos casos, complementarios. La tabla siguiente resume sus principales diferencias:

En resumen, **MediaPipe** resulta más adecuado para aplicaciones centradas en la estimación de características humanas y análisis del movimiento, mientras que **YOLO** se utiliza para la detección y clasificación de objetos en escenas más amplias. En conjunto, ambas herramientas pueden integrarse en sistemas híbridos donde se necesite combinar información semántica (qué objetos hay) y estructural (cómo se mueven o se relacionan las personas).

Framework de servidor: evaluación y elección

Para la capa de servidor del proyecto se necesita un API HTTP que reciba imágenes y parámetros, coordine los módulos de IA desarrollados

en Python y asegure que podamos añadir después funciones de seguridad (autenticación, trazabilidad, CORS). Bajo estos objetivos, se han analizado tres frameworks.

FastAPI (Python). **Ventajas:** permite programación asíncrona, genera automáticamente documentación interactiva, asegura tipos de datos con Pydantic. **Inconvenientes:** añade más estructura y definición de datos de la necesaria al inicio. **Motivo de descarte:** se buscaba avanzar rápido con una API sencilla y sin tanta formalidad estructural.

Django REST Framework (Python). **Ventajas:** completo para proyectos grandes, con gestores, serializadores y autenticación ya integrada. **Inconvenientes:** demasiado pesado para un backend cuyo foco principal es orquestar inferencias de IA. **Motivo de descarte:** la complejidad no se ajusta al alcance de este TFG.

Express.js (Node.js). **Ventajas:** minimalista, gran comunidad. **Inconvenientes:** implicaría usar otro lenguaje además de Python, lo que complica la integración con los modelos de IA. **Motivo de descarte:** se opta por mantener todo el servidor en Python para mayor coherencia.

Framework elegido: Flask

Descripción. Flask es un microframework web en Python. Se caracteriza por ofrecer lo esencial —rutas, peticiones, respuestas— sin imponer una estructura compleja. Su diseño ligero permite añadir sólo lo que el proyecto necesite.

Ventajas para este proyecto.

- Permite lanzar rápidamente un endpoint que reciba la imagen y devuelva un resultado, sin código innecesario.
- Al estar en Python facilita conectar directamente con los módulos de IA.
- Su simplicidad permite que, cuando lo necesitemos, añadamos seguridad, control de acceso o logging sin tener que rehacer la arquitectura.
- Los desarrollos iniciales son rápidos y el despliegue sencillo.

Justificación de elección. Dado que el servidor funciona como una capa de orquestación —no como un sistema transaccional complejo— se requiere algo ligero, sencillo de mantener y que permita crecer cuando haga falta. Flask cumple claramente esos requisitos: arranque rápido, bajo coste inicial, buena integración con Python y evolución controlada.

Funcionamiento en el proyecto. Se definirán rutas HTTP para subir imágenes o enviar parámetros, dichas rutas delegarán el trabajo a los módulos internos de IA y devolverán respuestas en formato JSON. La estructura del proyecto podrá separarse en rutas / servicios / módulos, lo que mantiene el código ordenado y escalable.

Librería para simulación de detecciones: evaluación y elección

Para validar tempranamente el *pipeline* del backend (subida de imagen → procesamiento simulado → respuesta con imagen anotada + JSON), se requiere una librería ligera que permita abrir imágenes, dibujar anotaciones sintéticas (cajas, texto) y devolver resultados sin integrar todavía modelos de IA reales. Bajo estos objetivos, se han analizado tres alternativas.

OpenCV (Python). **Ventajas:** muy completa para visión por computador; dispone de utilidades avanzadas de E/S, conversión de color y primitivas de dibujo.[8] **Inconvenientes:** instalación y dependencias más pesadas de lo necesario para una simulación inicial; curva de entrada mayor si sólo se requieren anotaciones simples. **Motivo de descarte:** se busca minimizar fricción y tiempo de puesta en marcha en el Sprint 1.

Matplotlib (Python). **Ventajas:** ampliamente conocida, permite renderizar y guardar imágenes con textos y formas.[3] **Inconvenientes:** está orientada a gráficos científicos, no al procesamiento de imágenes en un servicio web; el flujo para anotar y devolver imágenes resulta menos natural. **Motivo de descarte:** no optimiza el caso de uso de backend (anotar rápidamente y devolver).

Librería elegida: Pillow

Descripción. Pillow es el *fork* moderno de la Python Imaging Library (PIL). Facilita abrir, manipular y guardar imágenes, e incluye utilidades sencillas para dibujar rectángulos, textos y otras formas.[2]

Ventajas para este proyecto.

- **Ligereza y rapidez de integración:** instalación simple con `pip` y API directa para anotar imágenes.
- **Ajustada al caso de uso:** permite simular detecciones dibujando cajas y etiquetas sobre la imagen y devolverla junto con un JSON de “detecciones”.
- **Encaje con Flask:** fácil de integrar en endpoints que reciben ficheros y devuelven flujos binarios/imagen.
- **Evolución controlada:** cuando se integren modelos reales (YOLO, MediaPipe, etc.), se puede mantener la misma salida (imagen + JSON) sin cambios en el contrato.

Justificación de elección. Pillow ofrece exactamente lo necesario para simular resultados visuales y datos estructurados con un coste de adopción mínimo, evitando la complejidad innecesaria de alternativas más pesadas.

Funcionamiento en el proyecto. El endpoint del servidor (Flask) recibirá una imagen, generará detecciones sintéticas (clase, confianza, caja), anotará la imagen con Pillow y devolverá: (i) la imagen marcada y (ii) un JSON con la lista de detecciones. Este contrato será compatible con la futura sustitución de la simulación por modelos de IA reales.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Zachary J Bortolot and Randolph H Wynne. Estimating forest biomass using small footprint lidar data: An individual tree-based approach that incorporates training data. *ISPRS Journal of Photogrammetry and Remote Sensing*, 59(6):342–360, 2005.
- [2] Pillow Contributors. Pillow (pil fork) documentation, 2025. Consultado el 11 de noviembre de 2025.
- [3] Matplotlib Developers. Matplotlib documentation, 2025. Consultado el 11 de noviembre de 2025.
- [4] Google AI Edge. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Consultado en noviembre de 2025.
- [5] Google AI Edge. Repositorio oficial mediapipe en github. <https://github.com/google-ai-edge/mediapipe>, 2024. Consultado en noviembre de 2025.
- [6] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [7] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Eric Uboweja, Marcus Hays, Fan Zhang, Chuo-Ling Chang, Michael G Yong, Jiyong Lee, Wei-Ting Chang, Wan-Te Hua, Marion Georg, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [8] OpenCV Team. Opencv documentation, 2025. Consultado el 11 de noviembre de 2025.

- [9] Ultralytics. Repositorio oficial yolo en github. <https://github.com/ultralytics/ultralytics>, 2024. Consultado en noviembre de 2025.
- [10] Ultralytics. YOLOv8 documentation. <https://docs.ultralytics.com/models/yolov8/>, 2024. Consultado en noviembre de 2025.
- [11] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Internet; descargado 30-septiembre-2015].